

Tyler Mitchell

Software Engineer | Austin, TX

tyler.davis.mitchell@gmail.com
<https://github.com/tyler-mitchell>
<https://linkedin.com/in/tyler-d>

I build typed GPU runtimes, code intelligence for coding agents, and headless UI frameworks in TypeScript.

Experience

Founding Frontend Engineer | Federato

Dec 2021-Jan 2023

Rebuilt the frontend architecture while delivering the underwriting workbench for the company's first major customer.

- Established a shared monorepo, TypeScript adoption, stitched GraphQL code generation, and a fragment-based data layer.
- Built headless components and feature flags for multi-tenant configuration, and supported the team through mentoring and development standards.

Software Engineer | PayPal

Jan 2020-Jan 2022

Built account creation and onboarding flows on the Guest & Signup checkout team.

- Delivered passwordless onboarding and integrated Buy Now Pay Later into the guest checkout application.
- Contributed developer-experience improvements to the team's core application.

Software Engineer · Contract | UT Health San Antonio

Started 2020

Led development of patient management and surgery scheduling software for clinical teams.

- Owned architecture, design, support, and maintenance, with integrations for Epic, Microsoft Graph, and Qgenda.

Education

Bachelor of Computer Science | University of Texas at San Antonio
2017-2020

Skills

Core: TypeScript, React, WebGPU / WGSL, TypeGPU, React Three Fiber / Three.js, Node.js, GraphQL

Agent tooling: MCP / WebMCP, Codex, Claude Code, Effect, ArkType

Applications: Legend State, SurrealDB, PostgreSQL, Tailwind, Radix / React Aria, Docker, Figma

Interests: Real-time simulation, Procedural systems, Game design, Input ergonomics and hardware

Selected projects

WebGPU Simulation Engine

2026-present | Private

A deterministic browser runtime for physics, learning, and generated motion.

Typed execution: Fixed-capacity resources and phased operations let a device-independent compiler check access, aliasing, producer order, and phase placement before GPU execution. A timeline records steps and restores recorded positions through replay.

Physics and learning: GPU rigid-body physics combines an AVBD solver, contact generation, runtime joints, and kinematic input. A tensor IR produces forward and reverse GPU programs with memory reuse, Adam, gradient clipping, and checkpoints.

Motion and rendering: A browser motion model generates poses for physics and skinning. Rendering uses cached bundles, indirect and instanced draws, GPU culling, depth pyramids, occlusion, and glTF skin admission.

TypeScript | WebGPU | WGSL | TypeGPU | Effect

Agent-Directed Motion Studio

2026-present | Open source

Agents author human motion, actors, and cameras in a browser physics scene.

Shared editing: Agents and people use the same ArkType-validated operations for scenes, stories, actors, obstacles, cameras, and playback. Undo, authorship, and SurrealDB WASM persistence share that operation path.

Generated performance: The simulation engine generates motion on the GPU. Each actor drives a physics body, and GPU forward kinematics produces skin palettes from interpolated poses.

TypeScript | WebGPU | WebMCP | SurrealDB

Live project: <https://hyphenated-motion.netlify.app>

Source: <https://github.com/tyler-mitchell/hyphenated>

Code Intelligence for Agents

2026-present | Open source

Type Atlas gives coding agents direct access to the TypeScript language service.

Semantic navigation: Definitions, references, callers, and inferred types resolve to real symbols. Natural-language search finds code by behavior across monorepos.

Model-oriented output: Results are labeled and grouped for agent context windows. Bodies fold when signatures are sufficient, and type errors appear with ordinary navigation responses.

TypeScript | MCP | LSP | Node.js

Package: <https://npmjs.com/package/@type-atlas/mcp>

Source: <https://github.com/tyler-mitchell/type-atlas>

Spatial Window Manager

2026-present | Open source

A headless React library for opening, focusing, snapping, grouping, and docking windows on an infinite canvas.

One command path: Pointer, keyboard, UI, and automation use the same commands. The core consists of pure functions over plain data, with undo and redo at the document boundary.

Layered rendering: DOM windows and optional WebGPU or React Three Fiber scenes share the same camera. This portfolio uses the window manager for its board.

TypeScript | React | WebGPU

Package: <https://npmjs.com/package/@hyphenated/infinite-canvas>

Source: <https://github.com/tyler-mitchell/infinite-canvas>

Declarative Command Model

2026-present | Open source

One command declaration produces typed functions, a CLI, an MCP server, and a machine-readable specification.

Shared contracts: ArkType schemas define command inputs. Effect owns runtime and process execution. Help and shell completion come from the same declaration used by people and agents.

TypeScript | MCP | ArkType | Effect

Source: <https://github.com/tyler-mitchell/cmd-mesh>

Programmatic Package Management

2026-present · In development | Open source

A common API to install, uninstall, detect, and import packages across npm, Yarn, Bun, and pnpm.

TypeScript | Node.js

Source: <https://github.com/tyler-mitchell/package-management>